

DEXTERSQL: Deep Schema Exploration and Rule-based Correction for Text-to-SQL Generation

Anik Pramanik¹, Murat Kantarcioglu², Vincent Oria¹, Shantanu Sharma¹

¹New Jersey Institute of Technology, USA. ²Virginia Tech, USA.

Abstract

Prompting-based (*i.e.*, non-fine-tuning) Text-to-SQL methods, where underlying large language model parameters are not changed for the task, face three problems: (i) relying on coarse-grained schema information that may not reveal the fine-grained relationships needed to distinguish ambiguous columns, (ii) not capturing recurring SQL-generation failures, and (iii) suffering from omission, hallucination, or misplacement of conditions in complex questions.

This paper develops DEXTERSQL, a prompting/non-fine-tuning-based Text-to-SQL system that improves SQL generation with three novel components: (i) *deep schema explorer* that identifies ambiguous columns, analyzes their individual and joint data distributions to uncover their relationships and the distinct role of each, (ii) *database-agnostic rule creator* that mines mismatches between generated and gold SQL only on the training database and converts them into database-agnostic corrective rules that capture recurring LLM failure patterns; and (iii) *multi-path SQL generation* that introduces a dependency-tree-based intermediate representation that uses the question’s sentence structure to guide its decomposition into an SQL skeleton for final SQL generation.

DEXTERSQL achieves a higher accuracy compared to the state-of-the-art using both open-source/weight and closed-source/weight models. Particularly, DEXTERSQL’s shows a high improvement of at least 2.7% using an open-weight model (GPT-OSS-120B) on BIRD-Dev, with total accuracy 67.6%. DEXTERSQL also shows better improvement of at least 0.9% using closed-weight models, with total accuracy 71.6% and 72.2% on BIRD-Dev with GPT-4o and GPT-5.2.

1 Introduction

Text-to-SQL approaches translate a natural-language (NL) question into an executable SQL query for execution on a relational database. Early Text-to-SQL systems were based on task-specific neural architectures, *e.g.*, encoder-decoder models [39], grammar- and sketch-based decoders [36], and schema-aware encoders [32]. These systems were trained separately for each dataset [7, 9, 32, 39], and thus are less robust to unseen schemas, lack a general understanding of NL, and do not generalize on different datasets. Recently, large language models (LLMs) have substantially advanced Text-to-SQL process and surpassed these specialized models on standard benchmarks such as Spider [38] and BIRD [18].

LLM-based Text-to-SQL pipeline. LLM-based Text-to-SQL systems/approaches [2, 8, 13, 21–23, 31, 35] decompose the task into a sequence of steps, such as *schema linking* that narrows the full database schema to the tables and columns relevant to a NL question; *SQL generation* that produces one or more candidate SQL queries; *correction* that repairs invalid or inconsistent candidate SQL queries; and *selection* that selects a final SQL query from the candidate pool.

Fine-tuning vs. non-fine-tuning. LLM-based Text-to-SQL approach can be broadly categorized into two classes. *Fine-tuning (FT)* methods modifies an LLM’s weights on Text-to-SQL data by either training the generator directly or through auxiliary supervision [16, 17, 19, 24, 37]. *Non-fine-tuning (non-FT)* methods keep the LLM frozen and steer it purely through prompting, schema linking, NL decomposition, and correction [1–3, 5, 8, 11, 13, 23, 28, 29, 31, 35].

The two classes tradeoff differently. Fine-tuning can inject dataset- and schema-specific knowledge directly into the model to produce more accurate, self-contained models. However, this requires large amounts of labeled training data and must be repeated for every model or target domain. In contrast, non-fine-tuning methods require no training; they only manage the context and prompt provided to an off-the-shelf LLM. This makes them to improve automatically as base models advance, though their accuracy is bounded by the underlying LLM and is sensitive to the context they are given. These benefits, especially no training requirements, make non-fine-tuning methods easy for anyone to deploy.

DEXTERSQL. This paper *focuses on the non-fine-tuning setting*, due to the benefits offered by them and ease of deployment. However, despite rapid progress, such Text-to-SQL systems still face three major challenges.

(C1) *Detecting underlying database insights.* To produce a SQL query, LLM needs to understand the context from either the schema and/or sampled values (as in [13, 28, 35]). However, such information may not be sufficient to distinguish ambiguous/confusable columns, whose roles become clear only by analyzing their data distributions and relationships across tables.

(C2) *Capturing recurring model errors.* A Text-to-SQL system can have recurring SQL-generation failures due to the underlying LLM performance, so these reusable failure patterns must be separated from database-specific mistakes and converted into guidance for correcting future queries.

(C3) *effective SQL generation.* LLM-based SQL generation systems may omit, hallucinate, or misplace conditions and relationships in a complex NL question, producing executable SQL that answers an incomplete or different question.

We develop **DEXTERSQL**, a non-fine-tuning Text-to-SQL system that addresses the above-mentioned challenges. DEXTERSQL introduces three novel components, each addressing the above challenges: (i) *deep schema explorer*, an offline (*i.e.*, before arrival of any NL query) analysis to mine data-level relationships among ambiguous columns into compact disambiguation notes; (ii) *database-agnostic rule creator*, an offline process to learn recurring model failures using *only* the training dataset¹ and build database-agnostic correction rules that help for future NL questions; and (iii) *multi-path SQL generation*, which introduces a dependency-tree-based

¹NL queries are asked on a database, entitled *target database* that is completely disjoint from the training database.

Method	Pre-processing (§3.1)				Schema Linking (§3.2)	Text-to-SQL Generation (§3.3)				Post-processing (§3.4)		Accuracy	
	DB Value Matching (§3.1.2)	Additional Context (§3.1.1)	Deep Schema Exploration (§3.1.3)	Database-Agnostic Rule Creation (§3.1.4)		Ambiguity Resolver (§3.3.1)	Dependency Tree (§3.3.2)	Divide-and-Conquer (§3.3.2)	Few-shot Examples (§3.3.2)	Consistency (§3.4.2)	Rule-Based Correction (§3.4.1)	Spider-Test	BIRD-Dev
DAIL-SQL [8]	✗	✗	✗	✗	✗	✗	✗	✗	✓	✓	✗	74.0	51.8
C3 [5]	✗	✗	✗	✗	✓	✗	✗	✗	✓	✓	✗	69.3	53.8
Rethinking [21]	✓	✓	✗	✗	✓	✗	✗	✗	✗	✗	✗	72.2	54.6
DIN-SQL [23]	✗	✗	✗	✗	✓	✗	✗	✓	✓	✗	✗	70.0	56.2
AutoLink [33]	✓	✓	✗	✗	✓	✗	✗	✗	✗	✗	✗	77.5	57.4
OpenSearchSQL [35]	✓	✗	✗	✗	✓	✗	✗	✗	✓	✓	✗	67.8	58.2
RSL-SQL [2]	✓	✗	✗	✗	✓	✗	✗	✗	✗	✓	✗	74.6	59.3
Alpha-SQL [15]	✓	✗	✗	✗	✓	✗	✗	✓	✗	✓	✗	80.3	62.8
ApexSQL [1]	✓	✓	✗	✗	✓	✗	✗	✓	✗	✓	✗	79.1	64.2
DeepEye-SQL [12]	✓	✗	✗	✗	✓	✗	✗	✓	✓	✓	✗	81.9	64.9
DEXTERSQL	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	84.4	67.6

Table 1: Comparison of non-fine-tuning Text-to-SQL methods. Notations: *DB Value Matching*: looking up or embedding database cell values for grounding; *Additional Context*: extracting column statistics or metadata; *Deep Schema Exploration*: investigating data distributions and relationships between ambiguous columns; *Database-Agnostic Rule Creation*: mining recurring database-agnostic SQL-generation failures into reusable rules; *Schema Linking*: selecting the tables and columns relevant to a question; *Ambiguity Resolver*: selecting and incorporating guidance that distinguishes confusable columns; *Dependency Tree*: using dependency structure to guide decomposition into an intermediate SQL skeleton; *Divide-and-Conquer*: breaking a question into simpler sub-questions; *Few-shot Examples*: using retrieved examples as demonstrations; *Consistency*: selecting among multiple candidates through voting or agreement; and *Rule-Based Correction*: applying synthesized rules to detect and correct recurring SQL-generation failures.

intermediate representation to preserve question elements and combines it with few-shot in-context-learning and divide-and-conquer generation to produce multiple SQL candidates.

DEXTERSQL Performance. Text-to-SQL systems can be deployed using either open-weight, free LLMs (e.g., GPT-OSS-120B) or proprietary (closed-weight) LLMs (e.g., GPT-4o and GPT-5.2). These two classes of models show different tradeoffs. Open-weight models can be deployed within an organization’s local or private infrastructure, eliminating the need to transmit database schemas or other potentially sensitive information to third-party services. At sufficient scale, local deployment of smaller LLM models can also reduce marginal inference costs, and operational security and privacy. However, their SQL generation accuracy generally lags behind that of state-of-the-art proprietary LLMs. In contrast, proprietary models often provide stronger out-of-the-box performance, but they may require transmitting database schemas, metadata, or content to third-party services and can incur higher deployment and usage costs.

DEXTERSQL not only outperforms prior methods when using proprietary models, but also achieves substantially higher accuracy with open-weight models, making it particularly well suited for sensitive-data applications and local or cost-efficient deployment. Particularly, on BIRD-Dev benchmark [18], DEXTERSQL achieves an execution accuracy, EX (i.e., percentage of queries whose execution results match the gold-SQL results), of 67.6% using GPT-OSS-120B, outperforming existing non-fine-tuned Text-to-SQL systems by at least 2.7%; see the last column of Table 1. Also, DEXTERSQL achieves EX of 71.6% with GPT-4o and 72.2% with GPT-5.2, exceeding the state-of-the-art non-fine-tuning methods by at least 0.9% [1, 13].

2 DEXTERSQL

DEXTERSQL develops three novel components, each designed to tackle one of the key limitations of non-fine-tuned Text-to-SQL systems. Together, these components substantially improve SQL generation accuracy. This section provides contributions and high-level overview of DEXTERSQL.

2.1 Challenges and DEXTERSQL

To better understand what DEXTERSQL brings in, we organize our contributions around the above-mentioned three challenges and illustrate them using two example tables; see Tables 2 and 3, which are taken from the thrombosis_prediction database in BIRD [18].

Patient.Diagnosis stores the patient’s final diagnosis, whereas Examination.Diagnosis stores a preliminary diagnosis recorded for each examination. We use these tables to explain the three challenges and the corresponding DEXTERSQL components.

ID	Diagnosis
1	SLE
2	PSS

Table 2: Patient table.

ID	Diagnosis
1	SLE
1	PSS
2	SLE

Table 3: Examination table.

Challenge 1: Detecting underlying database insights. The first challenge is for an LLM to correctly select the desired tables and columns of a given database when an NL query is posted.

For instance, consider a question: “*what is the final diagnosis of the patient with ID 3?*” Answering it requires reading a Diagnosis column, but a surface-level (i.e., coarse-grain—examining only individual column values or relying only on schema) understanding of the schema cannot determine which Diagnosis column (Patient.Diagnosis or Examination.Diagnosis) should be used.

This example shows LLMs need to understand fine-grained details of the database (e.g., relationships between columns—analyzing how a column relates to its table or other tables through the values it contains) to overcome ambiguity in understanding the schema. This distinction is not obvious from the column names or from the overlapping values and can only be clear from the underlying data distribution: Patient.Diagnosis has one row per patient, whereas Examination.Diagnosis can have multiple rows for the same Patient.ID. Since Patient.Diagnosis records a single, final diagnosis per patient, it is the correct column for this patient-level question—a choice that requires inspecting the data distribution, not just the schema.

Existing prompt/non-fine-tuning-based approaches [1, 2, 5, 8, 11, 13, 21, 23, 28, 33, 35], however, explore databases at a surface level.

Our solution: Deep Schema Explorator (§3.1.3). DEXTERSQL identifies column pairs that are genuinely ambiguous for SQL generation. For each ambiguous pair, it examines the distribution within each column and joint distribution to measure how much the two columns’ value sets overlap. When their tables can be joined, DEXTERSQL additionally determines the fraction of records participating in the relationship and how often the ambiguous values

agree across joined records. An LLM summarizes this information into a concise disambiguation note explaining how the columns differ and when each should be used.

For example, Table 2 contains one diagnosis for patient 1, whereas Table 3 contains two examination diagnoses for the same patient. After joining on ID, one patient record therefore corresponds to multiple examination records, and the diagnosis values do not always agree: the patient diagnosis is SLE, while the examination diagnoses are SLE and PSS. Given this information, the LLM creates a note stating that Patient.Diagnosis represents the patient-level final diagnosis and should be used for final-diagnosis questions, whereas Examination.Diagnosis represents diagnoses associated with individual examination events.

Challenge 2: Capturing recurring model errors. The second challenge is to capture recurring SQL-generation failures of the underlying LLM.

For instance, consider a question: “What is the ratio of examinations diagnosed as PSS to examinations diagnosed as SLE?” The intended ratio is $1/2 = 0.5$. However, an LLM may generate $\text{SUM}(\text{Diagnosis} = \text{'PSS'}) / \text{SUM}(\text{Diagnosis} = \text{'SLE'})$, which may return 0 due to integer division. The correct formulation needs to produce a floating output via $\text{CAST}(\text{SUM}(\text{Diagnosis} = \text{'PSS'}) \text{ AS REAL}) / \text{SUM}(\text{Diagnosis} = \text{'SLE'})$, which returns 0.5.

This example shows that while an LLM generates an executable SQL that uses the correct tables and columns, but still returns the wrong result due to missing a SQL-formulation detail, e.g., integer versus floating-point division. Avoiding this error requires the LLM to recognize that a ratio needs floating-point division and insert a type conversion.

Existing prompt-based approaches use few-shot example selection [5, 8, 13, 28, 35] that select examples based on overall NL-question similarity, without isolating the phrases, SQL operators, or query constructs responsible for a recurring error. Hence, a retrieved example may appear relevant at a high level while lacking the specific SQL formulation needed to avoid the failure.

Our solution: Rule Creation (§3.1.4). DEXTERSQL uses *training data*² to mine recurring SQL-generation failures and convert them into database-agnostic rules for correcting generated SQL. First, it generates candidate SQL for sampled training questions and retains candidates whose results do not match the corresponding gold SQL. It then uses an LLM to explain the cause of each failure and removes explanations tied to a particular database, such as incorrect schema linking or misunderstanding a specific column. Next, it clusters the remaining database-agnostic explanations within and across training databases so that failures with the same underlying cause are grouped together. Finally, it uses each recurring failure group to create a *database-agnostic correction rule* that can be applied after SQL generation.

For the ratio example, the generated SQL returns 0 instead of the intended 0.5 because it divides two integer-valued aggregates without a floating-point conversion. From similar failures in the training data, DEXTERSQL creates a rule instructing the correction stage to cast one operand to a floating-point type before division.

²A training data contains NLQ and gold-SQL pair and is disjoint from testing data on which NL queries will be asked.

Notations	Meaning
$\mathcal{D}_{\text{target}}$	Target database over which the current NL question is answered
$\mathcal{D}_{\text{train}}$	Training database having NLQ–gold-SQL examples (used for rule creation)
$T_i.A_j[k]$	k^{th} cell value in a table T_i with a column T_i
$\text{NLQ} \& \text{gold}_{\text{SQL}}$	Natural-language question and gold SQL query for NLQ
$\text{Profile}_{T_i.A_j}$	Profile of column $T_i.A_j$
$\text{emb}_{T_i.A_j}$	Embedding of $\text{Profile}_{T_i.A_j}$
$\text{Index}_{\text{prof}}, \text{Index}_{\text{val}}$	Profile and value vector indices built during preprocessing
$\text{Token}(A)$	Normalized name tokens of column A
$\text{value}(A)$	Distinct non-null values of column A
$\text{Pair}_{\text{candidate}}$	Candidate ambiguous column-pair set
$\text{Pair}_{\text{confirm}}$	Confirmed ambiguous column-pair set
$\text{Note}_{(A_i.A_j)}$	Disambiguation note for column pair $\langle A_i, A_j \rangle$
$N_{\text{NLQ}} \& N_{\text{NLQ}}^*$	Candidate note and selected relevant note sets for NLQ
$\text{fail}_{\text{candidate}}, \text{fail}_{\text{final}}$	Candidate failure explanations and final database-agnostic failures
ErrorGroup	Set of dominant groups of similar failures
Rule	Complete set of synthesized correction rules
rule_i	An individual correction rule in Rule
$\text{Focused}_{\text{NLQ}}$	Final focused schema for NLQ
$\text{candidates}_{\text{SQL}}$	A generated candidate SQL
$\text{revises}_{\text{SQL}}$	Candidate SQL after LLM-based revision
$\text{correct}_{\text{SQL}}$	Set of SQL candidates after rule-guided correction
$\text{Cluster}_i, s_i$	Execution-result cluster and its representative SQL candidate
$\text{final}_{\text{SQL}}$	Final SQL query returned by DEXTERSQL

Table 4: Notations used in the paper.

When this failure pattern appears in a generated query, DEXTERSQL selects the rule and applies its correction.

Challenge 3: Effective multi-path SQL generation. The third challenge is preserving every required condition and relationship when decomposing a complex NL question for SQL generation. A free-form LLM decomposition may omit, hallucinate, or misplace a question element, producing executable SQL that answers an incomplete or different version of the question.

For example, “How many examinations for patient ID 1 were diagnosed as PSS?” contains two required filters: ID = 1 and Diagnosis = ‘PSS’. An LLM-generated decomposition may retain the patient filter but drop the diagnosis filter, producing a query equivalent to $\text{COUNT}(\ast) \text{ WHERE ID} = 1$. Using Table 3, this query returns 2 instead of the correct answer 1 because it counts both examination records for patient 1.

Prior approaches already use decomposition and multi-path reasoning to diversify SQL generation [13, 22, 23]. However, when a path relies on free-form LLM decomposition, its intermediate steps can still omit or hallucinate a required question element.

Our solution: Multi-path SQL generation (§3.3.2). DEXTERSQL introduces a *dependency-tree-based SQL* intermediate representation. Unlike prior approaches that rely on an LLM to generate the decomposition itself, DEXTERSQL deterministically derives the decomposition from the question’s dependency structure and converts it into an SQL-oriented intermediate representation. The obtained dependency tree preserves question’s mentions, literals, and their relationships in an SQL skeleton before outputting the final SQL.

To further increase candidate diversity, DEXTERSQL combines this new dependency-tree-based method with two established generation methods: few-shot in-context learning, which uses structurally similar training examples, and divide-and-conquer generation, which solves complex questions by breaking them into simpler sub-questions. These three paths provide complementary ways to generate SQL, making the overall process more effective when one reasoning strategy fails.

For the example above, the dependency tree separately identifies the phrases specifying patient ID=1 and diagnosis PSS. It maps them

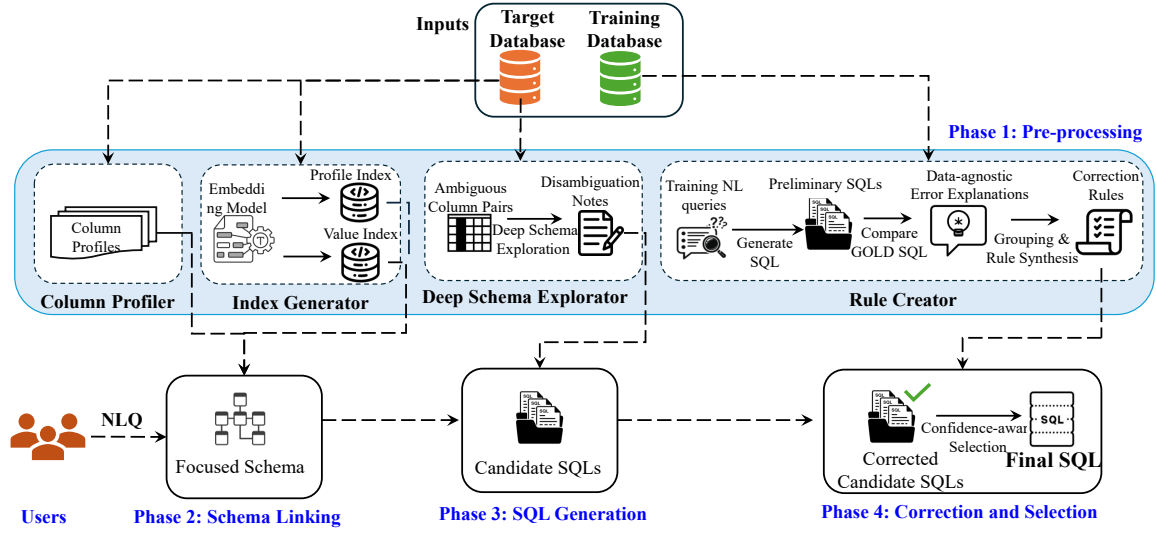


Figure 1: Overview of DEXTERSQL. Column Profiler, Index Generator, and Deep Schema Explorer process target databases to produce profiles, indices, and disambiguation notes, while Rule Creator mines training databases for correction rules. These artifacts are reused during online Schema Linking, SQL Generation, Correction, and Selection without updating model parameters.

to the two required filters, ID=1 and Diagnosis='PSS', in the intermediate SQL representation. The final SQL is then generated from this representation, reducing the risk that either filter is omitted.

2.2 High-Level Overview of DEXTERSQL

DEXTERSQL is divided into four phases, see Figure 1. This section provides an outline of four phases, which are then explained in detail in §3. We first define the terminology used in this paper.

We refer to the reference SQL query provided by a benchmark together with its corresponding natural language (NL) question as the **gold SQL query** (denoted by $gold_{SQL}$). We refer to the database on which future NL questions are answered as the **target database** (or **test database**), denoted by $\mathcal{D}_{target}$. DEXTERSQL may inspect only the target database’s schema, stored data, and developer-provided documentation. It does *NOT* access or use the target database’s gold SQL queries. We refer to a database containing NL questions paired with their corresponding gold SQL queries as a **training database**, $\mathcal{D}_{train}$. **Training databases and testing databases are disjoint.** Similar to other in-context Text-to-SQL systems (e.g., [8, 13, 35]), DEXTERSQL does not use the training database to update the LLM’s parameters or perform any fine-tuning.

DEXTERSQL separates execution into an **offline preparation** stage and an **online inference** stage. *Offline preparation* is performed before any NL question is received. It operates on both the training and target databases to construct the information required during inference. *Online inference* (or *SQL generation*) begins after an NL question over the target database is received. DEXTERSQL generates the corresponding SQL query, which is then compared with the benchmark’s gold SQL query to evaluate the system’s accuracy. Overall, PHASE 1 is executed once as an offline preprocessing step, whereas PHASES 2–4 are executed online for each incoming NL question over the target database.

PHASE 1: Preprocessing – An offline phase (§3.1). This phase runs *offline*, *only once*, before any NL question is asked. Inputs to this phase are $\mathcal{D}_{target}$ and $\mathcal{D}_{train}$, and the outputs are the per-column profiles, vector indices, disambiguation notes, and correction rules, which are reused throughout SQL query generation.

PHASE 1 has four components: (i) *Column profiler* that analyzes contents of $\mathcal{D}_{target}$ to build a per-column statistics profile; (ii) *Index generator* that builds searchable vector indices over the column profiles and column values of $\mathcal{D}_{target}$, to help LLM in retrieving semantically relevant columns and values during inference; (iii) *Deep schema explorer* that identifies groups of ambiguous columns and probes $\mathcal{D}_{target}$ to produce compact disambiguation notes so that confusable columns can be told apart during generation; and (iv) *Rule creator* that mines the $\mathcal{D}_{train}$ for recurring, **database-agnostic** mismatches between generated and gold SQL and distills them into generalized correction rules to capture recurring LLM failures.

PHASE 2: Schema Linking – An online phase (§3.2). This phase runs for each NL question. The input to this phase is the NL question together with the full database schema of $\mathcal{D}_{target}$ and the profiles and vector indices produced by PHASE 1. The output is a *focused schema* that contains only the tables and columns relevant to the NL question. PHASE 2 consist of a single component: *schema linker* that takes the input and produces the output, as mentioned before.

PHASE 3: SQL Generation – An online phase (§3.3). This phase runs for each NL question. The input to this phase is the NL question, the focused schema from PHASE 2, and the disambiguation notes from PHASE 1. The output is a set of candidate SQL queries. PHASE 3 has two components: (i) *Note incorporator* that inserts the only relevant disambiguation notes created by the deep schema explorer in PHASE 1 into the prompt for SQL generation; and (ii) *SQL generator* that produces candidate SQLs based on the focused schema

produced by PHASE 2 and the selected disambiguation notes, generating candidates with three reasoning strategies: dependency-tree-based intermediate-representation generation, few-shot in-context-learning generation, and divide-and-conquer generation.

PHASE 4: SQL Correction and Selection – An online phase (§3.4). This phase runs for each NL question. The input to this phase is the set of candidate SQL queries produced by PHASE 3. The output is a single final SQL query. This phase has two components: (i) *Correction* that repairs generated candidates using deterministic checks, execution feedback, and the database-agnostic correction rules produced in PHASE 1; and (ii) *Selection* that chooses the most reliable SQL from the corrected candidate pool.

3 Details of DEXTERSQL

This section explains each phase of DEXTERSQL and its components.

3.1 PHASE 1: Pre-processing

PHASE 1 consists of four components (see Figure 1): Column Profiler (§3.1.1), Index Generator (§3.1.2), Deep Schema Explorer (§3.1.3), and Rule Creator (§3.1.4); as discussed in §2.2. Below, we explain each component.

3.1.1 Column Profiler. Purpose. For a given NL question, during SQL generation based only on the database schema, an LLM cannot understand the underlying semantics of the database for three reasons: (i) the schema carries limited information about the data, (ii) column identifiers are often abbreviated or ambiguous and declared datatypes (e.g., TEXT) reveal little about the actual format or domain of the stored values, and (iii) a database may contain columns with similar names or related meanings, making it harder to identify the correct columns during schema understanding and SQL generation. To solve these problems, DEXTERSQL provides the LLM with fine-grained, data-derived information about each column through column profiler.

Working of column profiler. For each column A_j of each table T_i (i.e., $T_i.A_j$) of the *target database*, the column profiler builds a profile, $Profile_{T_i.A_j}$, from the column’s data, containing:

- (1) **Cardinality statistics:** include the row count, the number of null and non-null values, the number of distinct values, the range of values, and the minimum/ maximum values. These indicate whether a column is sparse (many nulls), unique or key-like (distinct count close to the row count), categorical (few distinct values), or numeric within a bounded range.
- (2) **Representative values:** a sample of the column’s values, together with its most frequent values and their counts. These give LLM concrete examples of what the column actually stores and how its values are formatted.

When available, $Profile_{T_i.A_j}$ also contains developer-provided column descriptions, value notes, and any other human provided documentation in a single profile.

3.1.2 Index Generator. Purpose. To map an NL query to columns, DEXTERSQL needs to quickly (i) retrieve semantically relevant columns whose profiles match the NL question, and (ii) locate columns that contain specific names, categories, dates, or numeric values from the NL question. To do so, DEXTERSQL builds two complementary indices over the *target database*—a profile index

$Index_{prof}$ for semantic retrieval over column profiles and a value index $Index_{val}$ for literal-value retrieval over column values.

Working of index generator. The two indices are created as follows:

- (1) **Profile index.** Index generator embeds, for each column $T_i.A_j$, its profile $Profile_{T_i.A_j}$ into a dense vector $emb_{T_i.A_j} = \text{Embed}(Profile_{T_i.A_j})$ using an embedding model and stores it in a FAISS vector index [6], resulting in profile index: $Index_{prof} = \{emb_{T_i.A_j} : T_i.A_j \in \mathcal{D}_{target}\}$.
- (2) **Value vector index.** This step builds cosine-similarity vector index [27] on distinct non-null values in each *text-typed* column. Text values are indexed because semantic matching can connect an NL phrase to a differently worded database value, whereas numeric values are handled via exact values and ranges. To bound the cost, at most η (10,000 in our system) values per column are embedded, normalized, and stored in $Index_{val}$.

Together, the profiles $Profile_{T_i.A_j}$ and the indices $Index_{prof}$ and $Index_{val}$ form the precomputed artifacts that DEXTERSQL uses during PHASE 1 schema linking (§3.2).

3.1.3 Deep Schema Explorer. Purpose. A key limitation of existing LLM-based Text-to-SQL systems during SQL query generation for a given NL query is the lack of exploratory information about how columns relate to one another to the underlying LLM (as we discussed in Challenge 1; see §2.1).

A trivial solution is to copy all column values into the LLM context. However, it is neither practical nor helpful (specifically for ambiguous columns) and introduces noise. To address this, DEXTERSQL identifies only column pairs that may be ambiguous and. For each such pair $\langle A_i, A_j \rangle$, DEXTERSQL produces a disambiguation note $Note_{\langle A_i, A_j \rangle}$ (as the output of this component) that characterizes how the two columns differ and when each should be used, (supplied as context for relevant questions during SQL generation).

Working of deep schema explorer. To uncover unclear relationships between columns, Deep Schema Explorer follows a four-step process executed over the *target database*. STEP 1 detects candidate column pairs that may be ambiguous to an LLM and outputs $Pair_{candidate}$. STEP 2 filters these candidates using LLM triage and outputs the confirmed ambiguous set $Pair_{confirm}$. STEP 3 investigates each confirmed pair by determining individual column-level and joint column-level statistics and producing evidence on the relationship of confirmed ambiguous column pairs. STEP 4 summarizes evidence from STEP 3 into an instruction-oriented disambiguation note $Note_{\langle A_i, A_j \rangle}$ that instructs SQL-generating LLM how the columns differ and when each one should be used. Below, we provide details of each step.

STEP 1: CANDIDATE GENERATION. Purpose. Since deep investigation over all possible column pairs is infeasible for large databases, we need cheap deterministic signals to identify only plausible ambiguous pairs.

Method. The input to this step is the target database schema together with the column profiles $Profile_{T_i.A_j}$ produced by the column profiler (§3.1.1), and the output is the candidate pair set $Pair_{candidate}$, which is provided to STEP 2. $Pair_{candidate}$ is constructed using two deterministic signals (rather than exhaustively investigating every column pair):

- (1) **Checking name similarity:** a pair $\langle A_i, A_j \rangle$ is added if the column names match or if their normalized name-token sets $Token(A_i)$ and

$\text{Token}(A_j)$ have high overlap, where $\text{Token}(A)$ denotes the set of normalized tokens [26] obtained from column name A .

- (2) **Checking profile similarity:** using $\text{Profile}_{T_i.A_i}$ and $\text{Profile}_{T_j.A_j}$ from §3.1.1, now, we embed each profile and add the pair when the cosine similarity between the two profile embeddings exceeds a threshold. Finally, this step removes structurally obvious non-ambiguities, such as declared primary-key/foreign-key pairs and identifier columns. The remaining pairs form $\text{Pair}_{\text{candidate}}$.

Example. In the running example in §2.1, Examination.Diagnosis and Patient.Diagnosis share the same column name, so name similarity adds $\langle \text{Examination.Diagnosis}, \text{Patient.Diagnosis} \rangle$ to $\text{Pair}_{\text{candidate}}$. In contrast, obvious identifier pairs such as Examination.ID and Patient.ID are removed because their relationship is already explained by the primary-key/foreign-key structure.

STEP 2: LLM TRIAGE. *Purpose.* The candidate set $\text{Pair}_{\text{candidate}}$ may contain column pairs that look similar by name or profile but are not actually confusing to LLM for SQL generation. LLM triage filters these candidates and keeps only the genuinely ambiguous pairs.

Method. The inputs to this step are the candidate pair set $\text{Pair}_{\text{candidate}}$ from STEP 1, the column profiles produced by Column Profiler (§3.1.1), the database schema, including primary-key/foreign-key relationships, and any additional database context, e.g., subject-matter-expert metadata supplied with the benchmark. The output is the confirmed ambiguous set $\text{Pair}_{\text{confirm}} \subseteq \text{Pair}_{\text{candidate}}$, which is passed to STEP 3 for deep investigation.

For each pair $\langle A_i, A_j \rangle \in \text{Pair}_{\text{candidate}}$, DEXTERSQL uses an LLM-as-judge to decide whether a realistic NL question could plausibly refer to either column and lead the SQL-generating LLM to pick the wrong one. To make the judgment effective, DEXTERSQL evaluates each pair under three prompts with increasing levels of information: (i) the column profiles and sample values produced by Column Profiler; (ii) the profiles with relevant primary-key/foreign-key information; and (iii) the full schema, profiles, and additional database context. Each prompt returns a binary verdict, and DEXTERSQL takes a majority vote.

Example. For $\langle \text{Examination.Diagnosis}, \text{Patient.Diagnosis} \rangle$, DEXTERSQL asks the LLM judge the same ambiguity question under three prompt variants. In the first prompt, it shows only the two column profiles and sample values, such as SLE, and asks whether an NL question mentioning “diagnosis” could plausibly refer to either column. In the second prompt, it adds relevant primary-key/foreign-key context, so the judge can see that Examination.Diagnosis belongs to examination rows while Patient.Diagnosis belongs to patient rows, and asks whether this distinction is still easy for a SQL-generating LLM to confuse. In the third prompt, it shows the full schema with profiles and context and asks for the same binary ambiguity verdict in the complete database setting. Since the prompt variants agree that the pair can confuse SQL generation, the majority vote keeps it in $\text{Pair}_{\text{confirm}}$.

STEP 3: DEEP INVESTIGATION. *Purpose.* Given the final set of ambiguous pairs $\text{Pair}_{\text{confirm}}$, this step aims to understand the underlying relationship between each pair of columns — how the two columns’ values are distributed and how their distributions vary across the joined rows if there is a join path between their tables.

Method. The input to this step is the confirmed ambiguous set $\text{Pair}_{\text{confirm}}$ from STEP 2. For each pair of columns $\langle A_i, A_j \rangle \in$

$\text{Pair}_{\text{confirm}}$, the output is evidence describing their individual distributions and, when applicable, their distribution across a join path. These per-pair column evidence are provided to STEP 4.

To determine a relationship between each confirmed column pair $\langle A_i, A_j \rangle \in \text{Pair}_{\text{confirm}}$, where A_i belongs to table T_i and A_j belongs to table T_j , we find (i) the individual column data distributions and (ii) the join-conditioned data distribution across the columns when their tables can be connected through a join path, as follows:

- (1) **Determining data distribution within a column.** For each column A_* in a confirmed ambiguous pair, where $* \in \{i, j\}$, this step takes all statistics produced by Column Profiler (§3.1.1). Furthermore, for each confirmed ambiguous pair $\{A_i, A_j\}$, this step computes the distinct non-null value sets $\text{value}(A_i)$ and $\text{value}(A_j)$ and their Jaccard similarity:

$$\text{overlap}(A_i, A_j) = \frac{|\text{value}(A_i) \cap \text{value}(A_j)|}{|\text{value}(A_i) \cup \text{value}(A_j)|}.$$

For text columns, this step also computes a case-insensitive version of the overlap. These measurements show whether the columns use similar value domains despite their potentially different roles.

- (2) **Determining data distribution across columns.** This analysis is necessary because two columns may have similar names and value distributions, yet their semantic roles often become apparent only after examining how their values interact through PK/FK relationships. We use $T_i \bowtie_{T_i.X=T_j.X} T_j$ to denote a join between T_i and T_j based on a primary-key/foreign-key (PK/FK) relationship. To capture the relational characteristics of ambiguous columns, we compute three join-path measurements. These measurements quantify (i) how many rows from one table participate in the join, (ii) the average multiplicity of the join relationship, and (iii) how frequently the ambiguous column values agree after the join. Together, these signals help distinguish columns that appear similar when viewed independently but serve different semantic roles within the database schema.

We assume that $T_i.X$ is the primary key and $T_j.X$ is the corresponding foreign key. The same measurements are also computed when T_i and T_j are connected through one or more intermediate tables along a PK/FK join path.

Coverage measures the fraction of rows from T_i that participate in at least one join with T_j :

$$\text{coverage}(A_i, A_j) = \frac{|\pi_{T_i.X}(T_i \bowtie_{T_i.X=T_j.X} T_j)|}{|\pi_{T_i.X}(T_i)|}.$$

Fan-out measures, on average, how many joined rows are produced for each participating row from T_i . This captures the cardinality of the relationship and indicates whether a matched row in T_i typically corresponds to a single row or multiple rows in T_j :

$$\text{fanout}(A_i, A_j) = \frac{|T_i \bowtie_{T_i.X=T_j.X} T_j|}{|\pi_{T_i.X}(T_i \bowtie_{T_i.X=T_j.X} T_j)|}.$$

Agreement measures how frequently the two ambiguous columns contain identical values among the joined tuples. A high agreement score suggests that the columns may represent the same underlying concept despite appearing in different tables:

$$\text{agreement}(A_i, A_j) = \frac{|\sigma_{T_i.A_i=T_j.A_j}(T_i \bowtie_{T_i.X=T_j.X} T_j)|}{|T_i \bowtie_{T_i.X=T_j.X} T_j|}.$$

If no PK/FK join path exists between T_i and T_j , DEXTERSQL omits these join-path measurements and relies solely on the column-level distributional statistics described earlier.

Example. For the diagnosis pair, let $A_i = \text{Patient.Diagnosis}$ and $A_j = \text{Examination.Diagnosis}$. The column-level probe first compares their distributions, including nulls, frequent values, and value overlap, and finds that both columns share diagnosis labels such as SLE, PSS, and APS.

The join-path probe then finds a direct path $\text{Examination.ID} \rightarrow \text{Patient.ID}$. After joining through this path, it computes $\text{coverage}(A_i, A_j)$, $\text{fanout}(A_i, A_j)$, and $\text{agreement}(A_i, A_j)$ over the overlapping joined rows. These measurements show that the two columns share a diagnosis vocabulary, but $\text{Examination.Diagnosis}$ covers only a subset of patients and often disagrees with Patient.Diagnosis after joining. This evidence is passed to note synthesis, which summarizes that Patient.Diagnosis should be used for patient-level final-diagnosis questions, while $\text{Examination.Diagnosis}$ should be used for examination-level diagnosis questions.

STEP 4: NOTE SYNTHESIS. *Purpose.* The previous step produces useful but low-level evidence (not fine-grained enough to place directly in SQL generation prompt). This step converts the output of STEP 3 into concise guidance that LLM uses to disambiguate columns.

Method. Input to this step is the evidence computed in STEP 3 for each confirmed pair $\langle A_i, A_j \rangle$; and the output is a disambiguation note $\text{Note}_{\langle A_i, A_j \rangle}$ for each ambiguous pair.

For each pair $\langle A_i, A_j \rangle \in \text{Pair}_{\text{confirm}}$, this step uses an LLM to summarize the evidence obtained in STEP 3 into guidance $\text{Note}_{\langle A_i, A_j \rangle}$ that states how the columns differ, when each column should be used, and any other information needed to help the LLM make the correct decision during inference. If multiple ambiguous pairs share columns, this step groups them into a **confusable family** and creates one *family-level note*. The final output is a per-database noteset $\mathcal{N}(\mathcal{D}_{\text{target}})$, computed **once, offline** and reused during inference as discussed in the schema-linking phase (§3.2).

Example. For the diagnosis pair, note synthesis looks at the Step 3 evidence, including value overlap, coverage, fan-out, and agreement, and infers that the two columns share a diagnosis vocabulary but play different roles: Patient.Diagnosis is a patient-level final diagnosis, while $\text{Examination.Diagnosis}$ is an examination-level diagnosis tied to a specific examination event. To make this distinction clear during inference, the resulting $\text{Note}_{\langle A_i, A_j \rangle}$ describes the role of each column and gives usage guidance: use Patient.Diagnosis when the question asks for a patient’s diagnosis or final diagnosis, and use $\text{Examination.Diagnosis}$ when the question asks about diagnosis information associated with an examination.

3.1.4 Rule Creator. Purpose. To overcome recurring SQL-generation errors of the underlying LLM, DEXTERSQL creates rules using only training databases, as discussed in Challenge 2 (§2.1). These rules are database-agnostic and created only once, offline, without fine-tuning the model.

Working of rule creator. Rule Creator follows a four-step process. Step 1 samples a subset of training questions, mines generated SQLs that disagree with the gold SQL, and explains the observed failures. Step 2 filters these explanations to keep reusable database-agnostic failures. Step 3 clusters similar failures into dominant error groups. Step 4 converts each dominant group into a correction rule rule_i . The output is the rule set *Rule*, where each rule provides a directive indicating when it applies and how to fix the corresponding SQL error. We describe each step below.

STEP 1: ERROR MINING. Purpose. The objective of this step is to find concrete LLM failures on the training databases, which provide the raw material for rule creation.

Method. The inputs are training examples $\langle \mathcal{D}_{\text{train}}, \text{NLQ}, \text{gold}_{\text{SQL}} \rangle$ and an underlying LLM. The output is the candidate failure-explanation set $\text{fail}_{\text{candidate}}$.

This step samples questions associated with $\mathcal{D}_{\text{train}}$ using difficulty labels when available. If such labels are unavailable, this step estimates difficulty based on the gold SQL structure, such as whether the query contains joins, nesting, aggregation, grouping, ordering, or set operations, so that the sampled questions cover a range of SQL complexities.

Then, for each sampled question NLQ , this step provides the question, the schema context from $\mathcal{D}_{\text{train}}$, and any question-specific evidence to LLM, and asks it to generate multiple candidate SQLs. Each generated $\text{candidate}_{\text{SQL}}$ is executed on $\mathcal{D}_{\text{train}}$ and compared with gold_{SQL} . A candidate whose execution result matches gold_{SQL} is discarded because it answers the NL question correctly.

For each remaining incorrect candidate, DEXTERSQL uses an LLM to compare $\text{candidate}_{\text{SQL}}$ with gold_{SQL} under the relevant context and explain why the generated SQL fails. If one candidate contains multiple independent problems, DEXTERSQL keeps them as separate per-question failure explanations. Repeating this process for all sampled questions produces $\text{fail}_{\text{candidate}}$.

Example. For the Challenge 2 ratio question in the running example (§2.1), gold_{SQL} casts one side of the division to REAL and returns 0.5. The LLM may instead generate $\text{candidate}_{\text{SQL}}$ as $\text{SUM}(\text{Diagnosis} = \text{'PSS'}) / \text{SUM}(\text{Diagnosis} = \text{'SLE'})$ over Examination. Although it selects the correct table and filters, this candidate returns 0 because both operands are integer-valued. Its execution result therefore differs from that of gold_{SQL} , so DEXTERSQL retains the candidate and adds an explanation: The query omits a floating-point conversion, leaving both aggregate expressions as integers; SQLite therefore performs integer division and truncates the fractional result to 0.

STEP 2: DATABASE-AGNOSTIC ERROR ISOLATION. Purpose. Not every mismatch should be used to create a corrective rule. Some errors arise from missing schema information, incorrect schema linking, or a database-specific misunderstanding; such errors are tied to a particular database and do not generalize. This step therefore keeps only database-agnostic errors.

Method. The input to this step is $\text{fail}_{\text{candidate}}$ from STEP 1; the output is the final database-agnostic failure set $\text{fail}_{\text{final}} \subseteq \text{fail}_{\text{candidate}}$. For each explanation, DEXTERSQL uses an LLM to decide whether the failure reflects a reusable SQL-formulation issue rather than a database-specific artifact such as a particular table, column, literal, or schema-linking mistake. Explanations judged to be database-specific are discarded, while database-agnostic explanations are kept in $\text{fail}_{\text{final}}$ for clustering.

Example. The missing-REAL-cast explanation from STEP 1 is retained in $\text{fail}_{\text{final}}$: the same integer-division error can occur in any database whenever a ratio divides two integer aggregates. In contrast, consider a patient-diagnosis question whose gold SQL uses Patient.Diagnosis , but whose generated SQL uses $\text{Examination.Diagnosis}$. This mismatch arises from the meanings of these particular columns, so DEXTERSQL discards its explanation as a database-specific schema-linking error.

STEP 3: CLUSTERING ERRORS. *Purpose.* We need to cluster similar database-agnostic explanations to form dominant error groups.

Method. The input to this step is $fail_{final}$ from STEP 2; the output is $ErrorGroup$, the set of dominant groups in which similar failure explanations are clustered together. DEXTERSQL clusters the explanations hierarchically with an LLM. Within each training database, it divides the corresponding explanations into manageable batches and asks the LLM to group explanations that describe the same underlying SQL-formulation error. It then compares and merges similar batch-level groups to form one set of error groups for that database. DEXTERSQL repeats this comparison-and-merging process across training databases, merging groups that describe the same underlying error. Groups with too few supporting explanations are discarded, and the remaining dominant groups form $ErrorGroup$ for STEP 4.

Example. The Challenge 2 explanation, which identifies a missing floating-point conversion as the cause of a truncated ratio, is first grouped with similar explanations from thrombosis_prediction. During cross-database merging, this group is merged with a group from another database containing similar truncated-ratio errors.

STEP 4: RULE SYNTHESIS. *Purpose.* Rule synthesis converts each dominant error group into a reusable correction rule.

Method. The input is the dominant error group $ErrorGroup$ from STEP 3, and the output is the rule set $Rule$ containing correction rules $rule_i$, which are used during inference to identify LLM SQL failures and correct them. For each dominant error group in $ErrorGroup$, DEXTERSQL uses an LLM to synthesize a canonical rule:

$$rule_i = \langle gist, bad\text{-}pattern, correct\text{-}pattern, fix \rangle.$$

Here *gist* summarizes the recurring failure and the situation in which it occurs, *bad-pattern* identifies the faulty SQL formulation to detect, *correct-pattern* gives the corresponding correct formulation, and *fix* provides the instruction for transforming the faulty formulation into the correct one.

Example. From the truncated-ratio group, DEXTERSQL creates the following rule. *Gist:* A ratio formed by dividing integer counts or aggregates omits a floating-point conversion and may be truncated. *Bad pattern:* $SUM(cond_a) / SUM(cond_b)$. *Correct pattern:* $CAST(SUM(cond_a) AS REAL) / SUM(cond_b)$. *Fix:* When this pattern is used to compute a ratio, convert one operand to a floating-point value before division. The resulting $rule_i$ is added to $Rule$ for later SQL correction.

3.2 PHASE 2: Schema Linking

Purpose. Schema linking maps an NL question to the part of the database schema needed to answer it. This is necessary because real databases may contain many tables and columns, and giving the full schema to the LLM can introduce irrelevant columns, confuse similar schema elements, and make SQL generation less reliable. The goal is therefore to construct a focused schema $Focused_{NLQ}$ that preserves the tables and columns needed for the question while removing unnecessary schema context.

Working. Given an NL question NLQ over target database $\mathcal{D}_{target}$, the inputs to this phase are schema of $\mathcal{D}_{target}$, the column profiles $Profile_{Ti, Aj}$ produced by Column Profiler (§3.1.1), the profile and value indices $Index_{prof}$ and $Index_{val}$ produced by Index Generator

(§3.1.2), and any additional schema information, such as subject-matter-expert metadata supplied with the benchmark. The output is a focused schema, $Focused_{NLQ}$, containing the tables and columns needed to answer NLQ , which is provided to PHASE 3. DEXTERSQL constructs this output in two steps.

STEP 1: PRELIMINARY FOCUSED-SCHEMA CONSTRUCTION. *Purpose.* We need to prune several of the tables/columns at the coarse-grain level, since the schema of $\mathcal{D}_{target}$ may contain many irrelevant columns/tables to NLQ .

Method. All inputs to this phase are provided to the first step, and the output is a *preliminary focused schema* $Focused'_{NLQ}$.

This step selects columns by (i) searching $Index_{prof}$ for columns whose profile descriptions are semantically similar to NLQ — this retrieves columns that match the meaning of the question even when their names are not explicitly mentioned, and (ii) extracting literals from NLQ and searches $Index_{val}$ for similar values stored in $\mathcal{D}_{target}$ — value matching helps to select a (candidate) column containing that value.

The columns returned by this way with their tables form $Focused'_{NLQ}$. The retrieved database values are also retained as grounding information for SQL generation.

STEP 2: FINAL FOCUSED-SCHEMA CONSTRUCTION. *Purpose.* Profile similarity and literal-value matching in STEP 1 may miss required schema elements when they are only indirectly implied by NLQ or are needed solely to connect tables. They may also retrieve semantically similar columns that are not required by the intended SQL query. This step therefore uses the preliminary focused schema to guide bidirectional schema linking and determine the final set of tables and columns required to answer NLQ .

Method. The inputs are NLQ and the full schema of $\mathcal{D}_{target}$ supplied to this phase; $Focused'_{NLQ}$ produced in STEP 1; the column profiles $Profile_{Ti, Aj}$ produced by Column Profiler (§3.1.1); the value index $Index_{val}$ produced by Index Generator (§3.1.2); and any subject-matter-expert metadata supplied with the benchmark. The output is the final focused schema $Focused_{NLQ}$.

The idea of the step is to generate preliminary SQL queries using an LLM based on three different prompts having: $Focused'_{NLQ}$, full schema of $\mathcal{D}_{target}$, and $Profile_{Ti, Aj}$ with schema information. Then, based on preliminary SQL queries, this step identifies any missing columns or tables and adds to the preliminary focused schema, resulting in the final $Focused_{NLQ}$.³

3.3 PHASE 3: SQL Generation

PHASE 3 consists of two components: (i) *Note incorporator* (§3.3.1), which selects the question-relevant disambiguation notes $\mathcal{N}_{NLQ}^*$; and (ii) *SQL generator*, which produces candidate SQL queries using the focused schema $Focused_{NLQ}$ produced by PHASE 2 and the selected notes. We describe each component below.

3.3.1 Note Incorporator. *Purpose.* DEXTERSQL creates a per-database note set $\mathcal{N}(\mathcal{D}_{target})$ for ambiguous column pairs in STEP 4 of Deep Schema Explorer in PHASE 1 (§3.1.3). However, during SQL generation, only a small subset of these notes is relevant to NLQ , since injecting all notes would add noise and may confuse

³ [2, 28] inspired the process of schema linking.

the SQL-generating LLM. Note Incorporator, thus, selects only the notes needed for the current question.

Working. Given NLQ over target database $\mathcal{D}_{\text{target}}$, DEXTERSQL takes the focused schema $Focused_{NLQ}$ produced by PHASE 2 and the database note set $\mathcal{N}(\mathcal{D}_{\text{target}})$ produced by Deep Schema Explorer in PHASE 1 (§3.1.3).

It first filters $\mathcal{N}(\mathcal{D}_{\text{target}})$ to candidate notes $\mathcal{N}_{NLQ}$. A note $Note_{\langle A_i, A_j \rangle}$ is retained as a candidate when at least one of its columns, A_i or A_j , appears in $Focused_{NLQ}$. Then, this step asks an LLM to inspect NLQ , $Focused_{NLQ}$, and $\mathcal{N}_{NLQ}$ and decide which candidate notes are relevant to the question for preventing the SQL generator from choosing the wrong ambiguous column:

$$\mathcal{N}_{NLQ}^* = \text{LLM}(NLQ, Focused_{NLQ}, \mathcal{N}_{NLQ}) \subseteq \mathcal{N}_{NLQ}.$$

The LLM labels each candidate note as relevant or irrelevant to NLQ . Notes labeled irrelevant are removed from the candidate-note set – note that this operation does not remove any column from $Focused_{NLQ}$. The notes labeled ‘relevant’ make up the selected note set $\mathcal{N}_{NLQ}^*$ and are retained for prompt generation.

Furthermore, this step may expand $Focused_{NLQ}$ by adding new columns. Note that a note becomes a candidate whenever at least one of its two associated columns is already present in $Focused_{NLQ}$. However, a candidate note may contain another column that is not yet included in $Focused_{NLQ}$. In this case, the missing column is added to $Focused_{NLQ}$. Importantly, Note Incorporator only expands $Focused_{NLQ}$ and never removes any existing column. If $\mathcal{N}_{NLQ}^* = \emptyset$, no note is incorporated, and $Focused_{NLQ}$ remains unchanged.

Example. For the running example in §2.1, a question about a patient’s final diagnosis may initially include Examination.Diagnosis in $Focused_{NLQ}$ while omitting Patient.Diagnosis. Because one column in the pair is present, the note comparing the two diagnosis columns becomes a candidate. The LLM selects this note as relevant because it explains that Patient.Diagnosis is patient-level while Examination.Diagnosis is exam-level. DEXTERSQL then adds the missing Patient.Diagnosis column before SQL generation.

3.3.2 SQL Generator. Purpose. Different SQL questions may benefit from different reasoning processes, so relying on one generation path can cause all candidates to share the same failure. DEXTERSQL uses complementary generation paths to produce diverse candidates and increase the likelihood that at least one captures the intended SQL.⁴

Working. The inputs are NLQ ; the final focused schema $Focused_{NLQ}$ produced in STEP 2 and the matched database values retrieved in STEP 1 of PHASE 2 (§3.2); the disambiguation notes $\mathcal{N}_{NLQ}^*$ selected by Note Incorporator (§3.3.1); and any subject-matter-expert evidence supplied with the question in the benchmark. The output is a pool of candidate SQL queries, each denoted by $candidate_{SQL}$, which is provided to SQL Correction and Selection in PHASE 4 (§3.4).

DEXTERSQL provides these inputs to three generation paths. It combines a new dependency-tree-based generation strategy with two established strategies from prior work, as described below.

First, *dependency-tree-based intermediate-representation generation* captures the relationships among question elements before constructing SQL. To do so, DEXTERSQL first parses NLQ into a dependency tree. Figure 2 shows the tree, where each node contains

How many examinations for patient ID 1 were diagnosed as PSS?

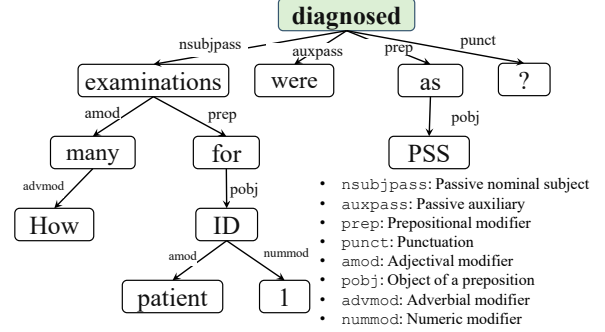


Figure 2: Dependency-tree example.

a question token and each labeled edge identifies its dependency on another token. DEXTERSQL maps the relevant parts of this tree to SQL components and combines them into a structured intermediate representation. The LLM then uses this representation to generate the final SQL while preserving the required question elements.

Second, *few-shot in-context-learning generation* uses similar training examples to demonstrate relevant question-to-SQL mappings. It retrieves structurally similar examples from the training set and provides them to the LLM as demonstrations.

Third, *divide-and-conquer generation* makes complex questions more manageable by breaking them into simpler sub-questions. It generates a partial SQL solution for each sub-question and composes them into a complete query.⁵

3.4 PHASE 4: SQL Correction and Selection

PHASE 4 consists of two components: (i) *correction*, which repairs candidate SQLs using deterministic checks, execution feedback, and the correction rules created by Rule Creator (§3.1.4); and (ii) *selection*, which chooses the most reliable candidate from the corrected pool. We describe each component below.

3.4.1 Correction. Purpose. LLMs may hallucinate schema elements, produce malformed SQL, or generate queries that fail during execution. Correction detects and repairs these problems before the final SQL selection.

Working. DEXTERSQL corrects each candidate in four steps. Step 1 creates a diagnostic report from deterministic syntax checking and execution feedback. Step 2 uses an LLM to revise the SQL using that report. Step 3 decides whether rule-based correction is needed and selects any relevant rules from Rule Creator in PHASE 1 (§3.1.4). Step 4 applies the selected rules, when any are selected, to produce the corrected candidate. The steps are described below.

STEP 1: DIAGNOSTIC REPORT CREATION. Purpose. This step finds syntax and execution problems caused by malformed clauses, hallucinated identifiers, unsupported syntax, or incorrect query logic.

Method. The inputs are a $candidate_{SQL}$ produced by SQL Generator in PHASE 3 (§3.3.2) and the target database $\mathcal{D}_{\text{target}}$, and the output is a diagnostic report for that candidate. This step first checks the candidate for syntax and parse errors using SQLGlot [20] which parses SQL into a dialect-aware abstract syntax tree and exposes

⁴This multi-path motivation is similar to CHASE-SQL [22].

⁵[8, 35] used few-shot in-context learning. [13, 22] used divide-and-conquer generation.

malformed clauses or unsupported syntax. Then, this step executes the candidate SQL and records execution feedback, such as runtime errors, empty results, or suspicious outputs. These syntax and execution signals are combined into a diagnostic report.

STEP 2: LLM REVISION. *Purpose.* This step repairs candidates using the diagnostic evidence from Step 1.

Method. The inputs are $candidate_{SQL}$ and its diagnostic report from STEP 1, together with NLQ , $Focused_{NLQ}$, the selected disambiguation notes N_{NLQ}^* from Note Incorporator (§3.3.1), and any subject-matter-expert evidence supplied with the question in the benchmark, and the output is the revised SQL $revises_{SQL}$. This step invokes an LLM in revision mode with these inputs so that it can repair syntax errors, execution failures, or suspicious outputs while preserving the intent of NLQ .

STEP 3: RULE RELEVANCE SELECTION. *Purpose.* This step determines whether the revised SQL still needs rule-based correction.

Method. The inputs are $revises_{SQL}$, NLQ , and the rule set $Rule$ produced by Rule Creator in PHASE 1 (§3.1.4), and the output is a (possibly empty) set of selected rules. This step parses and re-executes $revises_{SQL}$ to generate an updated diagnostic report. It then uses an LLM to decide whether rule-based correction is still needed and, if so, which $rule_i \in Rule$ apply, using the question, revised SQL, updated diagnostic report, and rule descriptions as context. If rule-based correction is not needed, no rule is selected.

STEP 4: RULE-GUIDED CORRECTION. *Purpose.* This step applies only the selected rules from the previous step to fix recurring SQL-generation errors.

Method. The inputs are $revises_{SQL}$ and the selected rules from STEP 3, and the output is a corrected SQL candidate.

If one or more rules are selected, this step uses them to correct the SQL. Each $rule_i$ provides a *gist*, *bad-pattern*, *correct-pattern*, and *fix* instruction (see STEP 4 of Rule Creator in §3.1.4), which guide the rewrite of $revises_{SQL}$. If no rule is selected, the revised SQL is kept unchanged. Applying this step to every revised candidate produces the corrected SQL candidate set $corrects_{SQL}$.

3.4.2 Selection. *Purpose.* Since correction produces multiple SQL candidates that may return different results, Selection chooses the most reliable candidate as the final SQL.

Working. The input is the corrected SQL candidate set $corrects_{SQL}$ from Correction (§3.4.1), where each candidate has passed through revision and rule-guided correction, and the output is $final_{SQL}$.

DEXTERSQL first executes every corrected candidate and clusters candidates that produce identical result sets. The clusters are ranked by size, and each top cluster, denoted by $Cluster_i$, is represented by one SQL candidate s_i . Then, DEXTERSQL assigns s_i an execution-confidence score equal to the fraction of corrected candidates that fall in its cluster, as: $Conf(s_i) = |Cluster_i|/|corrects_{SQL}|$.

If the execution confidence of the top-ranked representative s_i exceeds a threshold, DEXTERSQL treats the execution agreement as strong enough and returns that representative as $final_{SQL}$. Otherwise, DEXTERSQL selects representatives from the top- K clusters and asks an LLM to adjudicate them pairwise using NLQ , $Focused_{NLQ}$, the selected disambiguation notes N_{NLQ}^* from Note Incorporator (§3.3.1), any subject-matter-expert evidence supplied with the question in the benchmark, the candidate SQLs, and their

execution-confidence scores. During comparison, candidates supported by more SQLs are favored unless their logic conflicts with the NL question. DEXTERSQL then counts how often each candidate is preferred across the pairwise comparisons, combines this comparison support with its execution confidence, and returns the highest-scoring candidate as $final_{SQL}$.

4 Experimental Evaluation

This section experimentally compares DEXTERSQL against existing non-fine-tuning-based Text-to-SQL methods. We first provide details of the entire experimental setup and then the results.

4.1 Dataset, Setup, and Baselines

Datasets. We use two datasets: (i) *BIRD* [18] contains 12,751 NLQ-SQL pairs over 95 databases from more than 37 domains. In our experiments, 1,534 NLQ-SQL pairs over 11 databases in the full *BIRD-Dev* form $\mathcal{D}_{target}$, and 9,428 NLQ-SQL pairs over 69 databases in *BIRD-Train* form $\mathcal{D}_{train}$, of which randomly selected $\approx 3,000$ NLQ-SQL pairs form $\mathcal{D}_{train}$ for Rule Creator (§3.1.4). (ii) *Spider* [38] contains 10,181 NLQ-SQL pairs over 200 databases from 138 domains. In our experiments, 2,147 NLQ-SQL pairs over 40 databases in *Spider-Test* form $\mathcal{D}_{target}$, and 7,000 NLQ-SQL pairs over 140 databases in *Spider-Train* form $\mathcal{D}_{train}$, of which $\approx 3,000$ are only by Rule Creator.

Evaluation Metrics. We use the following three metrics in different experimental results:

- (1) **Execution Accuracy (EX):** considers a generated SQL as correct when its execution result matches that of the gold SQL. EX is used to evaluate SQL-generation performance in Exps. 1, 2, 4, 5, 6.
- (2) **Upper-bound Execution Accuracy (UB-EX):** finds the percentage of questions for which at least one generated candidate has the same execution result as the gold SQL (assuming an oracle always selects the correct candidate). UB-EX measures the maximum accuracy achievable from the candidate pool. Exp. 4 uses UB-EX also.
- (3) **Recall and Precision:** Recall measures the fraction of columns referenced by the gold SQL that are retained in the focused schema, while Precision measures the fraction of focused-schema columns referenced by the gold SQL. Exp. 3 uses recall and precision.
- (4) **Valid Efficiency Score (VES)** [18]: measures the execution efficiency of generated SQL queries that return the same results as the gold SQL. For N questions, it is computed as:

$$VES = \frac{1}{N} \sum_{i=1}^N EX\left(gold_{SQL}^{(i)}, final_{SQL}^{(i)}\right) \sqrt{\frac{t_{gold}^{(i)}}{t_{final}^{(i)}}}.$$

Here, $gold_{SQL}^{(i)}$ and $final_{SQL}^{(i)}$ denote the gold SQL and final SQL for the i th NLQ query, respectively. Here, $EX(\cdot, \cdot)$ denotes whether the execution results of the two SQL queries match, as defined in Execution Accuracy above. And, $t_{gold}^{(i)}$ and $t_{final}^{(i)}$ denote the execution times of the i th SQL query. Exp. 7 uses VES.

Setup. Experiments were conducted on an HPC cluster running Red Hat Enterprise Linux 9.6 with NVIDIA A100 80GB GPUs. To validate the effectiveness of DEXTERSQL, we use three distinct models: the open-weight GPT-OSS-120B model and the closed-weight GPT-4o and GPT-5.2 models. Furthermore, for retrieval during index construction in PHASE 1 (§3.1.2), we use Qwen3-Embedding-0.6B.

Method	Spider-Test	BIRD-Dev
DAIL-SQL [8]	74.0	51.8
C3 [5]	69.3	53.8
Rethinking Schema Linking [21]	72.2	54.6
DIN-SQL [23]	70.0	56.2
AutoLink [33]	77.5	57.4
OpenSearch-SQL [35]	67.8	58.2
RSL-SQL [2]	74.6	59.3
Alpha-SQL [15]	80.3	62.8
APEX-SQL [1]	79.1	64.2
DeepEye-SQL [13]	81.9	64.9
DEXTERSQL	84.4	67.6

Table 5: Exp. 1: Execution accuracy of non-fine-tuning methods using GPT-OSS-120B.

Baselines. Since DEXTERSQL is a non-fine-tuning framework, we compare it against **10 prompt-based**, training-free Text-to-SQL methods. These baselines are: DAIL-SQL [8], DIN-SQL [23], C3 [5], RSL-SQL [2], OpenSearch-SQL [35], Rethinking Schema Linking [21], Alpha-SQL [15], AutoLink [33], DeepEye-SQL [13], and ApexSQL [1].

4.2 Experimental Results

In this section, we evaluate the following:

- (1) SQL-generation accuracy with a open-weight model, Exp. 1 (§4.2.1).
- (2) SQL-generation accuracy on closed-weight models, Exp. 2 (§4.2.2).
- (3) Recall and precision of the final focused schema, Exp. 3 (§4.2.3).
- (4) Individual and combined contributions of SQL generation method/paths, Exp. 4 (§4.2.4).
- (5) Effects of the confidence threshold and SQL selection strategies, Exp. 5 (§4.2.5).
- (6) Contributions of Deep Schema Explorer, dependency-tree-based generation, and Rule Creator-based correction, Exp. 6 (§4.2.6).
- (7) Execution efficiency of correctly generated SQL queries, Exp. 7 (§4.2.7).

4.2.1 Exp. 1: SQL-Generation Accuracy with Open-Weight model. The goal of this experiment is to find DEXTERSQL performance using open-weight LLM model. We executed all baseline approaches, with available implementations, ourselves locally using GPT-OSS-120B, which activates about 5.1B parameters per token. Table 5 shows the results.

Findings. DEXTERSQL achieves the highest accuracy on both benchmarks, reaching 84.4% on Spider-Test and 67.6% on BIRD-Dev. On BIRD-Dev, the closest baseline is DeepEye-SQL at 64.9%, compared with 67.6% for DEXTERSQL, an improvement of 2.7% by our system, and the same holds on Spider-Test — DeepEye-SQL is again the closest baseline at 81.9%, while DEXTERSQL reaches 84.4%, showing an improvement of 2.5%.

4.2.2 Exp. 2: SQL-Generation Accuracy across Closed-Weight Models. The goal of this experiment is to compare SQL-generation accuracy of DEXTERSQL against the baseline approaches using closed-weight models: GPT-4, GPT-4o, and GPT-5.2.

EX on GPT-4 and GPT-4o (Table 6): For this experiment, we included the published works⁶ that reported accuracy using GPT-4 and 4o. Also, we selected *only DeepEye-SQL in this experiment* to run ourselves locally, since results using GPT-OSS-120B in Table 5 show that DeepEye-SQL is a close competitor to DEXTERSQL. So, we executed DeepEye-SQL locally on GPT-4o ourselves. On such closed-weight models, DEXTERSQL also achieves the highest EX at 71.6%, exceeding the strongest baseline that is APEX-SQL, by 0.9%.

EX on GPT-5.2 (Table 7): For GPT-5.2, we select only APEX-SQL and DeepEye-SQL because the GPT-4/GPT-4o results in Table 6 show that they are the two closest competitors to DEXTERSQL. Table 7 shows that DEXTERSQL achieves 72.2% EX, exceeding APEX-SQL at 69.7% and DeepEye-SQL at 69.3%. Thus, DEXTERSQL outperforms the strongest baseline by 2.5 percentage points.⁷

EX on other models. Some systems report results with other LLMs. Table 8 compares DEXTERSQL with CHESS and AutoLink using Gemini-1.5-Pro and DSR-SQL using DeepSeek-V3.1, based on the numbers reported in their respective papers. The strongest of these systems is AutoLink at 68.7%, which DEXTERSQL exceeds by 2.9%.

Findings. DEXTERSQL consistently outperforms existing non-fine-tuning Text-to-SQL systems with different closed-weight models.

4.2.3 Exp. 3: Evaluating Schema-Linking Accuracy. The goal of this experiment is to find whether DEXTERSQL retains the schema elements required for SQL generation while excluding irrelevant schema context more effectively than existing training-free schema-linking methods. Following prior schema-linking evaluations [1], we evaluate schema linking on BIRD-Dev using recall and precision, as defined in Evaluation Metrics (§4.1).

For DEXTERSQL, we measure recall and precision over the final focused schema used for SQL generation with GPT-OSS-120B. This schema is produced by the schema-linking phase (§3.2) and then enriched by Note incorporator (§3.3.1), which uses the outputs of Deep Schema Explorer (§3.1.3). Recall that when a relevant ambiguity note identifies an ambiguous column missing from the initial focused schema, DEXTERSQL adds that column before SQL generation — recovering required ambiguous columns that schema linking alone may omit, while the *note-relevance filtering limits unnecessary additions to preserve precision*.

Table 9 reports the recall and precision of DEXTERSQL and five *training-free schema-linking methods* on the same BIRD-Dev set. DEXTERSQL achieves 97.09% recall and 72.26% precision, exceeding the strongest baseline, APEX-SQL, by 0.94% and 4.21%, respectively. This shows that incorporating relevant findings from Deep Schema Explorer improves required-column coverage while limiting irrelevant schema context.

Findings. Among the baseline approaches with schema linking, DEXTERSQL achieves the highest recall and precision, reaching 97.09% recall and 72.26% precision — showing that DEXTERSQL retains more columns required by the gold SQL while introducing less irrelevant schema context than the other approaches.

⁶Published in a conference or a journal only, or provided the code.

⁷Although the APEX-SQL paper reports 70.7% with GPT-4o, running the same released APEX-SQL code with GPT-5.2 in our setting produces a lower accuracy of 69.7%.

Method	LLM	Simple	Moderate	Challenging	Total
C3 [5]	GPT-4	58.9	38.5	31.9	50.2
DAIL-SQL [8]	GPT-4	62.5	43.2	37.5	54.3
TA-SQL [25]	GPT-4	63.1	48.6	36.1	56.2
MAG-SQL [34]	GPT-4	65.9	46.2	41.0	57.6
SuperSQL [14]	GPT-4	66.9	46.5	43.8	58.5
MAC-SQL [31]	GPT-4	65.7	52.7	40.3	59.4
MCS-SQL [11]	GPT-4	70.4	53.1	51.4	63.4
RSL-SQL [2]	GPT-4o	74.4	57.1	53.8	67.2
OpenSearch-SQL [35]	GPT-4o	–	–	–	69.3
DeepEye-SQL [13]	GPT-4o	75.7	61.6	59.3	69.9
APEX-SQL [1]	GPT-4o	75.9	64.4	57.2	70.7
DEXTERSQL	GPT-4o	77.5	62.9	61.1	71.6

Table 6: Exp. 2: Execution accuracy on BIRD-Dev using GPT-4 or GPT-4o.

Method	LLM	Simple	Moderate	Challenging	Overall
DeepEye-SQL [13]	GPT-5.2	75.1	61.0	58.7	69.3
APEX-SQL [1]	GPT-5.2	74.9	63.4	56.2	69.7
DEXTERSQL	GPT-5.2	78.2	63.6	61.8	72.2

Table 7: Exp. 2: EX on BIRD-Dev using GPT-5.2.

Method	LLM	BIRD-Dev EX (%)
CHESS [30]	Gemini-1.5-Pro	68.3
DSR-SQL [10]	DeepSeek-V3.1	68.3
AutoLink [33]	Gemini-1.5-Pro	68.7
DEXTERSQL	GPT-4o	71.6

Table 8: Exp. 2: Comparison with non-fine-tuning methods using different frontier models on BIRD-Dev.

Active SQL generation paths	EX (%)	UB-EX (%)
Dependency-tree-based	66.3	72.1
Few-shot in-context learning	66.0	71.6
Divide-and-conquer	65.8	71.3
All three paths	67.6	74.8

Table 10: Exp. 4: Individual and combined performance of the SQL generation paths on BIRD-Dev using GPT-OSS-120B.

Method on BIRD-Dev	Recall (%)	Precision (%)
ReFoRCE [4]	51.29	68.05
AutoLink [33]	74.39	16.27
DeepEye-SQL [13]	95.40	49.5
RSL-SQL [2]	95.75	47.90
APEX-SQL [1]	96.15	51.95
DEXTERSQL	97.09	72.26

Table 9: Exp. 3: Schema-linking recall & precision.

4.2.4 Exp. 4: Evaluating Contributions of the SQL Generation Methods. The goal of this experiment is to measure the impact of our dependency-tree-based SQL generation method (§3.3.2) and to find the benefits of combining it with the established few-shot and divide-and-conquer methods. We evaluate each method individually, then all three together on BIRD-Dev with GPT-OSS-120B. All other pipeline components remain unchanged. For each configuration, we report EX and UB-EX (as defined in §4.1).

Table 10 shows that the dependency-tree-based method/path is the strongest individual generator, achieving 66.3% EX and 72.1% UB-EX. The two methods slightly lower EX of 66.0% for few-shot generation and 65.8% for divide-and-conquer generation. Combining them with the dependency-tree-based method raises EX to

67.6% and UB-EX to 74.8%, gains of 1.3% and 2.7% over the strongest individual path.

Figure 3 shows whether the three generation methods succeed on the same or different questions. In each panel, a circle represents the questions answered correctly by one method, and an overlapping region represents questions answered correctly by multiple methods. In EX analysis, all three methods answer 61.3% of questions correctly, while only 7.6% are answered correctly by one or two methods. In UB-EX analysis, all three methods generate at least one correct candidate for 66.0% of questions, while one or two methods generate a correct candidate for an additional 8.8%. Together, these regions produce the combined UB-EX of 74.8%, showing that the methods are complementary rather than redundant.

Findings. The dependency-tree-based method is the strongest individual SQL generator, achieving 66.3% EX and 72.1% UB-EX. Combining all three methods increases EX and UB-EX, showing that the few-shot and divide-and-conquer methods provide complementary candidates.

4.2.5 Exp. 5: Evaluating Confidence-Aware SQL Selection. The goal of this experiment is to find whether agreement among generated SQL candidates can identify when LLM review is needed

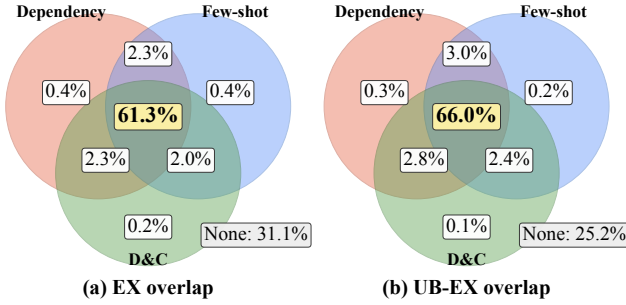


Figure 3: Correctness overlap among dependency-tree-based, few-shot in-context-learning, and divide-and-conquer SQL generation on BIRD-Dev using GPT-OSS-120B: (a) EX overlap and (b) UB-EX overlap.

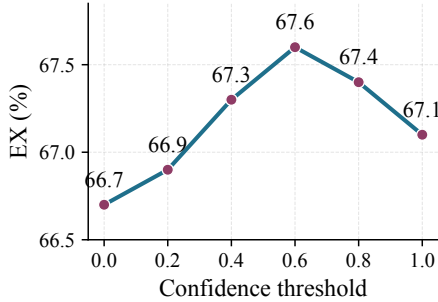


Figure 4: Execution accuracy across confidence shortcut thresholds on BIRD-Dev using GPT-OSS-120B.

during selection. In PHASE 4 (§3.4), DEXTERSQL executes the corrected candidates and groups those producing the same result. If the fraction of candidates in the largest group exceeds a threshold, their agreement is considered sufficient, and the group’s representative SQL is selected directly. Otherwise, DEXTERSQL uses an LLM to compare representatives from the top groups pairwise in a tournament-based selection. We evaluate this process on BIRD-Dev with GPT-OSS-120B by varying the threshold to find how effectively it separates cases that can rely on candidate agreement from those requiring LLM review.

We vary the threshold from 0 to 1 to control when LLM review is used. Figure 4 shows EX rises from 66.7% at threshold 0 to 67.6% at 0.6, then falls to 67.1% at threshold 1. These results show that selectively invoking LLM review when candidate agreement is insufficient improves SQL selection accuracy, with the highest EX achieved at a threshold of 0.6.

Findings. At a threshold of 0.6, confidence-aware selection achieves the highest EX of 67.6%. This confirms that using execution confidence to determine when LLM review is required improves SQL selection accuracy.

4.2.6 Exp. 6: Evaluating Contributions of the Novel Components. The goal of this experiment is to quantify the individual and combined contributions of DEXTERSQL’s three novel components:

Configuration	EX (%)
Full DEXTERSQL pipeline	67.6
w/o Dependency-tree-based generation (§3.3.2)	67.2
w/o Rule Creator-based correction (§3.1.4)	65.5
w/o Deep Schema Explorer (§3.1.3)	65.4
w/o all three novel components	63.3

Table 11: Exp. 6: Ablation results on BIRD-Dev using GPT-OSS-120B. “w/o all three novel components” disables Deep Schema Explorer, dependency-tree-based intermediate-representation generation, and Rule Creator-based correction simultaneously. Darker green indicates higher EX.

Deep Schema Explorer (§3.1.3), Rule Creator-based correction (§3.1.4), and dependency-tree-based intermediate-representation generation (§3.3.2). We conduct the ablation on BIRD-Dev with GPT-OSS-120B, disabling one component at a time.

Since Note Incorporator uses the notes produced by Deep Schema Explorer, disabling the latter also disables Note Incorporator. Similarly, disabling Rule Creator disables rule-based correction. Removing dependency-tree-based generation leaves the other two generation methods (i.e., few-shot and divide-and-conquer) active. We also evaluate the effect of disabling all three together.

Table 11 shows the result. The full pipeline obtains 67.6% execution accuracy. Removing dependency-tree-based generation decreases accuracy to 67.2%, while removing Deep Schema Explorer or Rule Creator-based correction reduces it to 65.4% and 65.5%, respectively. Disabling all three novel components produces the largest decrease, down to 63.3% accuracy. These results show that the individual components provide benefits and are most effective when used together.

Findings. The full DEXTERSQL pipeline achieves 67.6% EX. Removing dependency-tree-based generation, Rule Creator-based correction, or Deep Schema Explorer reduces EX to 67.2%, 65.5%, and 65.4%, respectively, showing that each novel component contributes to accuracy.

4.2.7 Exp. 7: Evaluating SQL Execution Efficiency. The goal of this experiment is to determine the execution efficiency of the final SQL queries generated by DEXTERSQL, and this is measured using VFS, as discussed in §4.1.

We evaluate DEXTERSQL on BIRD-Dev with GPT-OSS-120B using VES, and report results for the simple, moderate, and challenging subsets and the complete BIRD-Dev set. Table 12 compares DEXTERSQL with methods already included in our evaluation for which directly comparable BIRD-Dev VES results are available in prior work [14]. DEXTERSQL achieves the highest VES overall and across all difficulty levels, reaching 74.89, 54.59, and 52.44 on the simple, moderate, and challenging subsets, respectively, and 66.70 overall. It exceeds the next-highest overall result by 3.85 points, demonstrating stronger SQL execution efficiency.

5 Conclusion

We developed DEXTERSQL, a non-fine-tuning Text-to-SQL system. DEXTERSQL introduces three novel components: *Deep Schema Explorer* to uncover the different roles of ambiguous columns and create reusable disambiguation notes, *Rule Creator* to convert recurring database-agnostic generation failures from training data

Method	Simple	Moderate	Challenging	All
C3-SQL [5]	59.82	41.68	31.93	51.70
DAIL-SQL [8]	65.04	43.35	39.33	56.05
SuperSQL [14]	69.75	50.55	49.08	61.99
APEX-SQL [1]	69.06	53.47	50.18	62.56
DeepEye-SQL [13]	69.04	54.13	51.27	62.85
DEXTERSQL	74.89	54.59	52.44	66.70

Table 12: Exp. 7: Valid Efficiency Score on BIRD-Dev across difficulty levels

into correction rules, and a *dependency-tree-based SQL generator* that uses the question structure to construct an intermediate SQL representation. Experimental results validate DEXTERSQL effectiveness in generating SQL queries, showing higher accuracy on both open-weight and closed-weight models.

Acknowledgments

We are thankful to New Jersey Institute of Technology High Performance Computing (HPC) facility for providing the computational resources used in this work. We also thank Arda Ayna, Aruntej Thummepally, and Vineet Vora for their valuable assistance.

References

- [1] Bowen Cao, Weibin Liao, Yushi Sun, Dong Fang, Haitao Li, and Wai Lam. 2026. APEX-SQL: Talking to the data via Agentic Exploration for Text-to-SQL. *arXiv preprint arXiv:2602.16720* (2026).
- [2] Zhenbiao Cao, Yuanlei Zheng, Zhihao Fan, Xiaojin Zhang, Wei Chen, and Xiang Bai. 2024. Rsl-sql: Robust schema linking in text-to-sql generation. *arXiv preprint arXiv:2411.00073* (2024).
- [3] Yeounoh Chung, Gaurav T Kakkar, Yu Gan, Brenton Milne, and Fatma Ozcan. 2025. Is long context all you need? leveraging LLM’s extended context for NL2SQL. *arXiv preprint arXiv:2501.12372* (2025).
- [4] Minghang Deng, Ashwin Ramachandran, Canwen Xu, Lanxiang Hu, Zhewei Yao, Anupam Datta, and Hao Zhang. 2025. ReFoRCE: a text-to-SQL agent with self-refinement, consensus enforcement, and column exploration. *arXiv preprint arXiv:2502.00675* (2025).
- [5] Xuemei Dong, Chao Zhang, Yuhang Ge, Yuren Mao, Yunjun Gao, Jinshu Lin, Dongfang Lou, et al. 2023. C3: Zero-shot text-to-sql with chatgpt. *arXiv preprint arXiv:2307.07306* (2023).
- [6] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. 2025. The faiss library. *IEEE Transactions on Big Data* (2025).
- [7] Yujian Gan, Xinyun Chen, Jinxia Xie, Matthew Purver, John R. Woodward, John H. Drake, and Qiaofu Zhang. 2021. Natural SQL: Making SQL Easier to Infer from Natural Language Specifications. *CoRR abs/2109.05153* (2021). [arXiv:2109.05153](https://arxiv.org/abs/2109.05153) <https://arxiv.org/abs/2109.05153>
- [8] Dawei Gao, Haibin Wang, Yaliang Li, Xiuyu Sun, Yichen Qian, Bolin Ding, and Jingren Zhou. 2023. Text-to-sql empowered by large language models: A benchmark evaluation. *arXiv preprint arXiv:2308.15363* (2023).
- [9] Jiaqi Guo, Zecheng Zhan, Yan Gao, Yan Xiao, Jian-Guang Lou, Ting Liu, and Dongmei Zhang. 2019. Towards Complex Text-to-SQL in Cross-Domain Database with Intermediate Representation. In *Proceedings of the 57th Conference of the Association for Computational Linguistics, ACL 2019, Florence, Italy, July 28- August 2, 2019, Volume 1: Long Papers*, Anna Korhonen, David R. Traum, and Lluís Màrquez (Eds.). Association for Computational Linguistics, 4524–4535. doi:10.18653/V1/P19-1444
- [10] Zhifeng Hao, Qibin Song, Ruichu Cai, and Boyan Xu. 2025. Text-to-SQL as Dual-State Reasoning: Integrating Adaptive Context and Progressive Generation. *arXiv preprint arXiv:2511.21402* (2025).
- [11] Dongjun Lee, Choongwon Park, Jaehyuk Kim, and Heesoo Park. 2025. MCS-SQL: Leveraging Multiple Prompts and Multiple-Choice Selection For Text-to-SQL Generation. In *Proceedings of the 31st International Conference on Computational Linguistics*, Owen Rambow, Leo Wanner, Marianna Apidianaki, Hend Al-Khalifa, Barbara Di Eugenio, and Steven Schockaert (Eds.). Association for Computational Linguistics, Abu Dhabi, UAE, 337–353. <https://aclanthology.org/2025.coling-main.24/>
- [12] Boyan Li, Chong Chen, ZhuJun Xue, Yinan Mei, and Yuyu Luo. 2025. DeepEye-SQL: A software-engineering-inspired text-to-sql framework. *arXiv preprint arXiv:2510.17586* (2025).
- [13] Boyan Li, Chong Chen, ZhuJun Xue, Yinan Mei, and Yuyu Luo. 2026. DeepEye-SQL: A software-engineering-inspired text-to-sql framework. *Proceedings of the ACM on Management of Data* 4, 3 (SIGMOD (2026)), 1–28.
- [14] Boyan Li, Yuyu Luo, Chengliang Chai, Guoliang Li, and Nan Tang. 2024. The Dawn of Natural Language to SQL: Are We Fully Ready? [Experiment, Analysis & Benchmark]. *Proc. VLDB Endow.* 17, 11 (2024), 3318–3331. doi:10.14778/3681954.3682003
- [15] Boyan Li, Jiayi Zhang, Ju Fan, Yanwei Xu, Chong Chen, Nan Tang, and Yuyu Luo. 2025. Alpha-SQL: Zero-Shot Text-to-SQL using Monte Carlo Tree Search. In *International Conference on Machine Learning*. PMLR, 36810–36830.
- [16] Haoyang Li, Shang Wu, Xiaokang Zhang, Xinmei Huang, Jing Zhang, Fuxin Jiang, Shuai Wang, Tieying Zhang, Jianjun Chen, Rui Shi, et al. 2025. Omnisql: Synthesizing high-quality text-to-sql data at scale. *arXiv preprint arXiv:2503.02240* (2025).
- [17] Haoyang Li, Jing Zhang, Hanbing Liu, Ju Fan, Xiaokang Zhang, Jun Zhu, Renjie Wei, Hongyan Pan, Cuiping Li, and Hong Chen. 2024. CodeS: Towards Building Open-source Language Models for Text-to-SQL. *Proc. ACM Manag. Data* 2, 3 (2024), 127. doi:10.1145/3654930
- [18] Jinyang Li, Binyuan Hui, Ge Qu, Jiaxi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Ruiying Geng, Nan Huo, Xuanhe Zhou, Chenhao Ma, Guoliang Li, Kevin Chen-Chuan Chang, Fei Huang, Reynold Cheng, and Yongbin Li. 2023. Can LLM Already Serve as A Database Interface? A Big Bench for Large-Scale Database Grounded Text-to-SQLs. (2023). http://papers.nips.cc/paper_files/paper/2023/hash/83fc8fab171036305bbd1d4b8cc0021-Abstract-Datasets_and_Benchmarks.html
- [19] Yifu Liu, Yin Zhu, Yingqi Gao, Zhiling Luo, Xiaoxia Li, Xiaorong Shi, Yuntao Hong, Jinyang Gao, Yu Li, Bolin Ding, et al. 2026. Xiyang-sql: A novel multi-generator framework for text-to-sql. *IEEE Transactions on Knowledge and Data Engineering* (2026).
- [20] Toby Mao and SQLGlue Contributors. 2026. SQLGlue: Python SQL Parser and Transpiler. <https://github.com/tobymao/sqlglue>. Software repository, accessed July 28, 2026.
- [21] Md Mahadi Hasan Nahid, Davood Rafiei, Weiwei Zhang, and Yong Zhang. 2026. Rethinking schema linking: A context-aware bidirectional retrieval approach for text-to-sql. In *Findings of the Association for Computational Linguistics: EACL 2026*. 4516–4546.
- [22] Mohammadreza Pourreza, Hailong Li, Ruoxi Sun, Yeounoh Chung, Shayan Talei, Gaurav Tarlok Kakkar, Yu Gan, Amin Saberi, Fatma Ozcan, and Serkan O Arik. 2024. Chase-sql: Multi-path reasoning and preference optimized candidate selection in text-to-sql. *arXiv preprint arXiv:2410.01943* (2024).
- [23] Mohammadreza Pourreza and Davood Rafiei. 2023. Din-sql: Decomposed in-context learning of text-to-sql with self-correction. *Advances in neural information processing systems* 36 (2023), 36339–36348.
- [24] Mohammadreza Pourreza and Davood Rafiei. 2024. DTS-SQL: Decomposed Text-to-SQL with Small Large Language Models. *EMNLP 2024 (2024)*, 8212–8220. doi:10.18653/V1/2024.FINDINGS-EMNLP.481
- [25] Ge Qu, Jinyang Li, Bowen Li, Bowen Qin, Nan Huo, Chenhao Ma, and Reynold Cheng. 2024. Before generation, align it! a novel and effective strategy for mitigating hallucinations in text-to-sql generation. In *Findings of the Association for Computational Linguistics: ACL 2024*. 5456–5471.
- [26] Abigail Rai and Samarjeet Borah. 2020. Study of various methods for tokenization. In *Applications of Internet of Things: Proceedings of ICCIoT 2020*. Springer, 193–200.
- [27] Nils Reimers and Iryna Gurevych. 2019. Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of the 2019 conference on empirical methods in natural language processing and the 9th international joint conference on natural language processing (EMNLP-IJCNLP)*. 3982–3992.
- [28] Vladislav Shkapenyuk, Divesh Srivastava, Theodore Johnson, and Parisa Ghane. 2025. Automatic metadata extraction for text-to-sql. *arXiv preprint arXiv:2505.19988* (2025).
- [29] Sai Ashish Somayajula, Marianne Menglin Liu, Chuan Lei, Fjona Parllaku, Daniel Garcia, Rongguang Wang, Syed Fahad Allam Shah, Ankan Bansal, Sujeeth Bharadwaj, Tao Sheng, et al. 2026. SOMA-SQL: Resolving Multi-Source Ambiguity in NL-to-SQL via Synthetic Log and Execution Probing. *arXiv preprint arXiv:2606.11424* (2026).
- [30] Shayan Taleai, Mohammadreza Pourreza, Yu-Chen Chang, Azalia Mirhoseini, and Amin Saberi. 2024. Chess: Contextual harnessing for efficient sql synthesis. *arXiv preprint arXiv:2405.16755* (2024).
- [31] Bing Wang, Changyu Ren, Jian Yang, Xinnian Liang, Jiaqi Bai, Linzheng Chai, Zhao Yan, Qian-Wen Zhang, Di Yin, Xing Sun, et al. 2025. Mac-sql: A multi-agent collaborative framework for text-to-sql. In *Proceedings of the 31st International Conference on Computational Linguistics*. 540–557.
- [32] Bailin Wang, Richard Shin, Xiaodong Liu, Oleksandr Polozov, and Matthew Richardson. 2020. RAT-SQL: Relation-Aware Schema Encoding and Linking

- for Text-to-SQL Parsers. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, ACL 2020, Online, July 5-10, 2020*, Dan Jurafsky, Joyce Chai, Natalie Schluter, and Joel R. Tetreault (Eds.). Association for Computational Linguistics, 7567–7578. doi:10.18653/V1/2020.ACL-MAIN.677
- [33] Ziyang Wang, Yuanlei Zheng, Zhenbiao Cao, Xiaojin Zhang, Zhongyu Wei, Pei Fu, Zhenbo Luo, Wei Chen, and Xiang Bai. 2026. Autolink: Autonomous schema exploration and expansion for scalable schema linking in text-to-sql at scale. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 40. 33809–33817.
 - [34] Wenxuan Xie, Gaochen Wu, and Bowen Zhou. 2024. Mag-sql: Multi-agent generative approach with soft schema linking and iterative sub-sql refinement for text-to-sql. *arXiv preprint arXiv:2408.07930* (2024).
 - [35] Xiangjin Xie, Guangwei Xu, Lingyan Zhao, and Ruijie Guo. 2025. Opensearch-sql: Enhancing text-to-sql with dynamic few-shot and consistency alignment. *Proceedings of the ACM on Management of Data* 3, 3 (2025), 1–24.
 - [36] Xiaojun Xu, Chang Liu, and Dawn Song. 2017. Sqlnet: Generating structured queries from natural language without reinforcement learning. *arXiv preprint arXiv:1711.04436* (2017).
 - [37] Jiaxi Yang, Binyuan Hui, Min Yang, Jian Yang, Junyang Lin, and Chang Zhou. 2024. Synthesizing Text-to-SQL Data from Weak and Strong LLMs. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2024, Bangkok, Thailand, August 11-16, 2024*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, 7864–7875. doi:10.18653/V1/2024.ACL-LONG.425
 - [38] Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, Zilin Zhang, and Dragomir R. Radev. 2018. Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. (2018), 3911–3921. doi:10.18653/V1/D18-1425
 - [39] Victor Zhong, Caiming Xiong, and Richard Socher. 2017. Seq2SQL: Generating Structured Queries from Natural Language using Reinforcement Learning. *CoRR* abs/1709.00103 (2017). arXiv:1709.00103 <http://arxiv.org/abs/1709.00103>